

AI-use log

Adib Wahid Quader
LT2326, Assignment 1

I used ChatGPT during this assignment. As I was still exploring a lot of concepts in the assignment and was doubtful in writing the structure of the functions and classes for the assignment, i had to resort to AI for such cases. The table lists, grouped and paraphrased, what I asked, why I asked it, and what I did with the answer.

Prompt(s)	Why?	Incorporated as...
Asked for the Part 1 code: For a helper, <code>is_cjk</code> , that checks whether a character is a Chinese character, to count Chinese tokens in the vocabulary.	To get a working tokenizer pipeline and the statistics the assignment asks for.	Used the code and ran it myself. <code>is_cjk</code> is used in the cell that counts single- and multi-character Chinese tokens.
Asked how to set up Part 2 so that it runs on the GPU server (choosing a GPU, checking that CUDA is available) and how to fix the random seeds (<code>random</code> , <code>numpy</code> , <code>torch</code>) and the data shuffle.	The assignment requires GPU training, and I wanted the runs to be reproducible.	Used as the settings cell and the seeding in <code>train_model</code> . I chose the GPU and the number of epochs.
Asked for the Transformer layer, <code>class Block(nn.Module)</code> , together with an explanation of its parts (attention, layer normalisation, residual connections, feed-forward network, causal mask).	To build the layer of the language model and understand how the parts fit together.	Used the class. I fixed my own mistake of putting <code>forward</code> outside the class and an indentation error.
Asked for <code>class TransformerLM</code> , the full model with token and position embeddings, a stack of blocks, and the output layer.	To get a decoder-only language model whose vocabulary size can be changed.	Used the class, and counted its parameters per tokenizer (input embeddings, output layer, total).
Asked for <code>get_batch</code> , how to get windows from the streams and what process/ideas to follow later on.	How model handles token.	Used the function and later on followed by <code>def evaluate</code> which also needed help from AI.
Asked for the vocabulary-allocation method (which language each token belongs to), the coverage statistics and the error-analysis cells.	To train and compare the three models fairly, and to analyse the results.	Used the code and ran everything myself. The 90% threshold for “language-specific” tokens is my choice.
Pasted three error messages (a missing history file, an <code>IndentationError</code> , and models without a <code>forward</code> method) and asked what caused them.	I could not find the cause myself.	Fixed the cells: moved <code>forward</code> into the classes, corrected the indentation, and ran the training cells before plotting.
Asked for help reading my results, which patterns to discuss, and for a summary of the findings.	To choose the most informative patterns for the analysis sections.	Used the suggestions and checked all numbers against my notebook output.

Prompt(s)	Why?	Incorporated as...
Asked for help for the error analysis and what to discuss.	To find mean BPC, sentences with long words and other informations.	Used the fundamentals for creating <code>def sentence_bpc</code> and applied the rest accordingly.
Asked AI to turn my writing into the ACL LaTeX template (including the Chinese examples and an appendix).	To get the report into the ACL format recommended in the assignment.	Pasted it into Overleaf, compiled it with XeLaTeX, and filled in my name, the repository link and the remaining markers.
Asked for drafts of the README, the checkpoint checklist and <code>requirements.md</code> (how to train, evaluate and reproduce, and where the models are stored).	To make sure the repository and the checkpoint cover everything the assignment asks for.	Used the drafts, checked the paths and numbers against my notebook, and polished the wording.

All numbers in the report come from my own runs. I have read the submitted code and can explain it.